instead, can’t offer better-than-linear worst-case running time.

Moreover, the data structure used will have to be re-created from scratch at each iteration of k-means’ main cycle, so we won’t be dealing with a dynamic dataset that would cause a k-d trees to become imbalanced over time.

The fact that we have to create a new dataset at each iteration, at the same time, is one of the biggest CONs of using this approach, because we will have to pay this extra price; also, we will need `O(k)` extra memory to store it: it won’t change the asymptotic memory print of the algorithm, but in practice it will be relevant, especially if `k`, the number of centroids, is high.

In most applications, though, it is reasonable to expect to have `k << n`: in other words, the number of centroids will be several orders of magnitude smaller than the number of points.

Listing 12.5 shows how we can change the pseudocode for method `classifyPoints` to use a k-d tree in place of brute-force search.

As you can see, the code is much shorter than the version in listing 12.3, because most of the complexity of the search is now encapsulated in the `KdTree` class.

Notice that since we would like to find out the index of the centroid closer to each point, we assume that the `KdTree` object can keep track of the indices of its points in the initialization array, and that we have a query method that